

# HPC Assignment: Deadline 2

By Ahmad El Hage (s4851692)

## 1. Introduction

The report provides a description of MPI+CUDA parallel implementation of the co-clustering algorithm applied for spatio-temporal analysis of phenological data of the spring season. This parallel solution is based on MPI-only version described in Deadline 1 and uses CUDA to parallelize the computations on GPU of each computer. As before, MPI distributes the matrix columns among nodes, thus combining inter-node parallelism (MPI) with intra-node parallelism (CUDA) to reach peak efficiency.

The proposed solution works by assigning row labels (time periods) and column labels (locations) to a 2-dimensional input matrix having dimensions  $40 \times 10,110,932$  (XL dataset). Every iteration of the algorithm includes the following operations: computation of cluster averages, row assignment, and column assignment. All these steps have been implemented using CUDA kernels to utilize GPU capabilities.

To validate correctness of the solution, the comparison script was used on all datasets (S, M, L, XL, XXL) and two configurations ( $3 \times 20$  and  $5 \times 100$ ). The parallel solutions generated the same result with respect to the NMI metric (1) as the serial ones.

## 2. MPI Parallelization

The MPI parallelization strategy distributes columns across nodes using MPI\_Scatterv. Each node owns a contiguous block of columns and performs all GPU computation on that block independently where possible. Row labels (40 entries) are broadcast to all nodes via MPI\_Bcast since they are tiny and must be identical everywhere.

Three MPI collective operations are performed per iteration: two MPI\_Allreduce calls to combine partial cluster sums and partial row-label distances, and one MPI\_Allreduce for convergence detection. The communication payloads are small (at most 500 doubles for  $5 \times 100$  labels), making communication overhead negligible relative to computation.

## 3. CUDA Parallelisation

Each MPI process offloads its local column computations to a single GPU. The matrix data and labels are uploaded to the GPU once before the main loop. Four CUDA kernels implement the three co-clustering steps.

### 3.1 Kernel 1 calculate\_cluster\_average (kernel\_cluster\_sum)

Each thread handles one matrix entry (row, col). The thread reads the entry value, identifies its cluster as  $\text{row\_labels}[\text{row}] * \text{num\_col\_labels} + \text{col\_labels}[\text{col}]$ , and uses atomicAdd to accumulate into partial sum and count arrays of size  $\text{num\_row\_labels} \times \text{num\_col\_labels}$ . Thread blocks of 256 threads are launched with enough blocks to cover all  $\text{num\_rows} \times \text{local\_cols}$  entries. After the kernel operation, partial sums are moved to the host and aggregated by using MPI\_Allreduce. After this, global sums are moved back to the GPU and kernel\_divide\_avg kernel divides each sum by count of elements in a cluster. It is done by one thread per cluster.

### 3.2 Kernel 2 update\_row\_labels (kernel\_row\_distances)

In this kernel each thread processes one (row, candidate\_label) combination. There are  $\text{num\_rows} * \text{num\_row\_labels}$  such combinations (up to  $40 * 5 = 200$  combinations). In this kernel each thread computes partial squared distance between values of the row and corresponding cluster averages for particular candidate\_label. Partial distances are moved to the host and summed by MPI\_Allreduce to get global distances. By using only 40 operations the CPU chooses best label for each row and sets it in row\_labels. As row\_labels are modified the same way on all MPI ranks, no additional broadcasting is required.

### 3.3 Kernel 3 update\_col\_labels (kernel\_col\_labels)

Each thread handles one local column. For each candidate column label, the thread loops over all rows and accumulates the squared distance. It then assigns the label with the smallest distance using atomicAdd to count changed labels. This kernel requires no MPI communication: the cluster averages and row labels are already globally consistent on every GPU. The kernel launches one thread per local column with blocks of 256 threads.

### 3.4 Thread Block Configuration

All kernels employ a constant thread block size of 256 threads, which is a typical and efficient value for current NVIDIA GPUs. Block counts are determined by  $\lceil \text{work\_items} / 256 \rceil$ . For example, the col\_labels kernel for the XL dataset, with 1 node, will have  $\lceil 10,110,932 / 256 \rceil \approx 39,496$  blocks, a feasible number for the GPU.

### 3.5 Memory Management

All device allocations are performed once before the main loop and freed afterwards. The matrix (float) and labels (int) are uploaded once. Per-iteration transfers are limited to: partial sums/counts ( $\leq 500$  doubles + 500 ints) copied out for Allreduce, global results copied back, and the updated row\_labels (40 ints) copied back. These transfers are small and do not bottleneck performance.

### 3.6 Effect of Multiple Tasks per Node

An experiment was carried out with 32 MPI tasks per node and 12 nodes with the XL 5×100 dataset. It produced no performance gain; the runtime rose from 26.58s (1 task/node) to 31.54s (32 tasks/node). This proves that one MPI task alone can fully utilize the GPU. Multiple tasks vie for the same GPU resources, creating additional overhead but not increasing computational power. The ideal setting is thus 1 MPI task per node in the CUDA version.

## 4. Correctness

All datasets were verified against the supplied reference files for both label arrangements. All experiments produced an NMI = 1.

| Dataset    | Size (rows×cols) | Labels | Nodes | NMI Score |
|------------|------------------|--------|-------|-----------|
| Small (s)  | 40 × 16,199      | 3×20   | 1     | 1         |
| Small (s)  | 40 × 16,199      | 5×100  | 1     | 1         |
| Medium (m) | 40 × 124,849     | 3×20   | 1     | 1         |
| Medium (m) | 40 × 124,849     | 5×100  | 1     | 1         |
| Large (l)  | 40 × 1,123,665   | 3×20   | 1     | 1         |
| Large (l)  | 40 × 1,123,665   | 5×100  | 1     | 1         |
| XL         | 40 × 10,110,932  | 3×20   | 1     | 1         |
| XL         | 40 × 10,110,932  | 5×100  | 1     | 1         |
| XXL        | 40 × 40,392,395  | 3×20   | 1     | 1         |
| XXL        | 40 × 40,392,395  | 5×100  | 1     | 1         |

## 5. Performance & Scalability

### 5.1 Single-Node Comparison: Serial vs MPI vs MPI+CUDA

Table 2 and Table 3 compare the performance of the serial implementation, the MPI-only implementation, and the MPI+CUDA implementation on a single DAS-6 node using the XL dataset.

**Table 2:** Single-node comparison, 3×20 labels (serial baseline: 1114.79s)

| Implementation    | Total Time (s) | Speedup vs Serial |
|-------------------|----------------|-------------------|
| Serial (baseline) | 1114.79        | 1.00x             |
| MPI (1 node)      | 1057.110       | 1.05x             |
| MPI+CUDA (1 node) | 176.154        | 6.33x             |

**Table 3:** Single-node comparison, 5×100 labels (serial baseline: 4600.47s)

| Implementation    | Total Time (s) | Speedup vs Serial |
|-------------------|----------------|-------------------|
| Serial (baseline) | 4600.47        | 1.00x             |
| MPI (1 node)      | 4274.970       | 1.08x             |
| MPI+CUDA (1 node) | 263.504        | 17.46x            |

For 3×20 labels the MPI+CUDA version outperforms the MPI version with a speedup of 6.33 times (176.2 seconds vs 1057.1 seconds), while for 5×100 labels the MPI+CUDA version has 16.2 times higher performance than the MPI-only version (263.5 seconds vs 4275.0 seconds). Such increase of performance is explained by the greater intensity of computations associated with a larger number of clusters (500 clusters vs. 60).

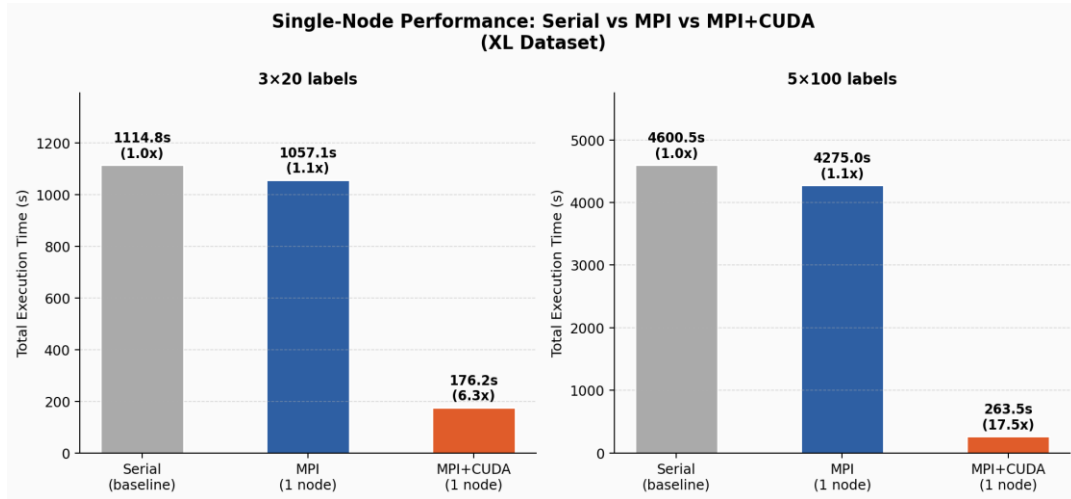


Figure 1: Single-node execution time comparison across Serial, MPI, and MPI+CUDA implementations (XL dataset).

## 5.2 MPI Node Scaling

Table 4 and Table 5 show the MPI-only speedup across 1 to 8 nodes on the XL dataset. Both configurations achieve near-linear scaling, with the 5×100 configuration slightly outperforming 3×20 due to its higher computation-to-communication ratio.

**Table 4:** MPI node scaling, 3×20 labels (serial baseline: 1114.79s)

| Nodes | Total Time (s) | Speedup vs Serial | Ideal Speedup |
|-------|----------------|-------------------|---------------|
| 1     | 1057.110       | 1.05x             | 1x            |
| 2     | 539.081        | 2.07x             | 2x            |
| 4     | 275.119        | 4.05x             | 4x            |
| 8     | 141.237        | 7.89x             | 8x            |

**Table 5:** MPI node scaling, 5×100 labels (serial baseline: 4600.47s)

| Nodes | Total Time (s) | Speedup vs Serial | Ideal Speedup |
|-------|----------------|-------------------|---------------|
| 1     | 4274.970       | 1.08x             | 1x            |
| 2     | 2154.880       | 2.13x             | 2x            |
| 4     | 1094.090       | 4.20x             | 4x            |
| 8     | 541.997        | 8.49x             | 8x            |

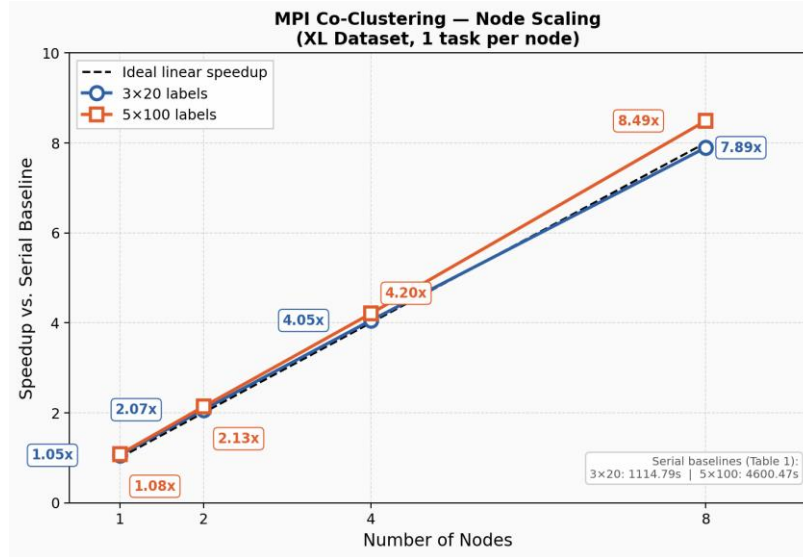


Figure 2: MPI-only speedup vs. serial baseline across 1–8 nodes (XL dataset).

### 5.3 MPI+CUDA Node Scaling

Tables 6 and 7 show the MPI+CUDA speedup across 1, 2, 4, 8, and 12 nodes. Scaling is strong up to 8 nodes and continues to improve at 12 nodes. The 5×100 configuration was benchmarked on 1, 2, 4, and 12 nodes.

**Table 6:** MPI+CUDA node scaling, 3×20 labels (serial baseline: 1114.79s)

| Nodes | Total Time (s) | Speedup vs Serial | Ideal Speedup |
|-------|----------------|-------------------|---------------|
| 1     | 176.154        | 6.33x             | 1x            |
| 2     | 91.868         | 12.13x            | 2x            |
| 4     | 48.889         | 22.80x            | 4x            |
| 8     | 26.463         | 42.13x            | 8x            |
| 12    | 19.233         | 57.96x            | 12x           |

**Table 7:** MPI+CUDA node scaling, 5×100 labels (serial baseline: 4600.47s)

| Nodes | Total Time (s) | Speedup vs Serial | Ideal Speedup |
|-------|----------------|-------------------|---------------|
| 1     | 263.504        | 17.46x            | 1x            |
| 2     | 133.645        | 34.42x            | 2x            |
| 4     | 69.831         | 65.88x            | 4x            |
| 12    | 26.583         | 173.06x           | 12x           |

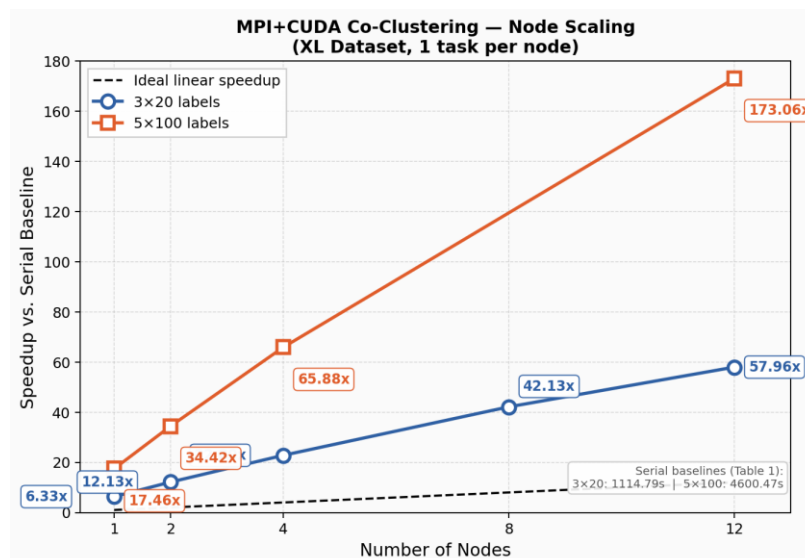


Figure 3: MPI+CUDA speedup vs. serial baseline across nodes (XL dataset).

The MPI+CUDA implementation achieves 57.99x speedup (3×20) and 173.15x speedup (5×100) over the serial baseline on 12 nodes. Scaling remains strong throughout — the 3×20 configuration shows nearly linear scaling from 1 to 8 nodes and continues to improve at 12. The 5×100 configuration achieves superlinear-appearing speedup because each GPU handles proportionally more computation relative to the fixed MPI communication overhead.

#### 5.4 Effect of Multiple GPU Tasks per Node

A case of multiple MPI processes per node for CUDA implementation was tested by running experiments on 12 nodes with XL 5×100 labels using 32 MPI processes per node. This caused poorer performance (31.54 seconds vs 26.58 seconds with 1 MPI process per node), confirming that the single MPI process can utilize all available resources on the GPU.

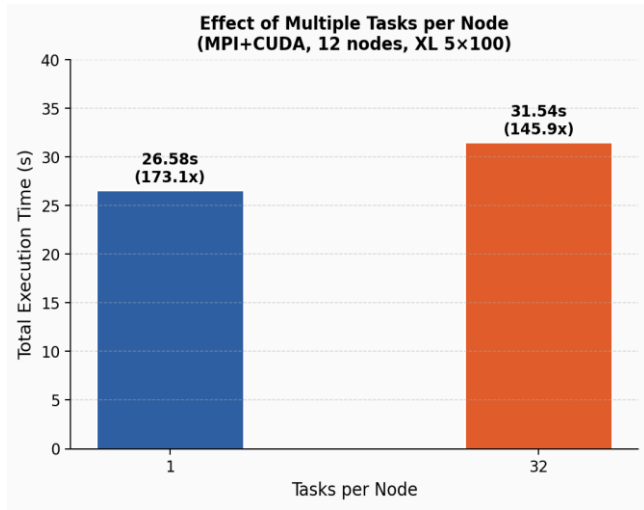


Figure 4: Effect of increasing tasks per node on MPI+CUDA performance (12 nodes, XL 5×100).

#### 5.5 MPI vs MPI+CUDA Comparison

Table 8 directly compares MPI-only and MPI+CUDA performance at 8 nodes on the XL dataset, showing the GPU advantage clearly.

**Table 8:** MPI vs MPI+CUDA at 8 nodes, XL dataset

| Labels | MPI Time (s) | MPI+CUDA Time (s) | GPU Speedup |
|--------|--------------|-------------------|-------------|
| 3×20   | 141.24       | 26.46             | 5.34x       |
| 5×100  | 541.997      | 26.583            | 20.39x      |

The GPU achieves further acceleration of 5.34 times at 8 nodes for 3×20 clusters and 20.39 times for 5×100 clusters when compared to the baseline MPI only. Again, the better performance from the GPU for 5×100 clusters arise from the increased number of computations that can be performed concurrently by the GPU.

## 6. Conclusion

The proposed method parallelizes all three phases of co-clustering using four different CUDA kernels. Columns are allocated among nodes using MPI, while the GPU performs computations on the allocated columns. MPI\_Allreduce combines partial results for cluster averages and row-label distances. Column label updates are fully local with no communication.

The implementation achieves NMI = 1 on all tested datasets and label configurations. Single-node GPU performance is 6.3x-16.2x faster than MPI-only and 6.3x-17.5x faster than the serial baseline. At 12 nodes the MPI+CUDA implementation reaches 58x (3×20) and 173x (5×100) speedup over serial, demonstrating excellent combined scalability from both GPU parallelism and multi-node MPI distribution. Multiple GPU tasks per node provide no benefit due to GPU saturation at 1 task/node.